

RAG General Knowledge Chatbot

A Locally-Hosted, Source-Confirmed Retrieval-Augmented Generation System

Under the guidance of: Pooja Shastri

Project Report

Repository: github.com/Khagendra-12/RAG-GK-ChatBot-Atul

Remote Internship Project

Table of Contents

Table of Contents.....	2
1. Project Overview	3
1.1 High-Level Architecture	3
1.2 Technology Stack	3
2. Backend Pipeline in Detail	4
2.1 Query Interpretation (query_interpret.py)	4
2.2 Web Search (search.py)	4
2.3 Retrieval & Reranking (retrieval.py)	4
2.4 Answer Generation (llm.py)	5
2.5 Caching and Chat History (cache.py, chat_history.py).....	5
3. Model Selection Process.....	5
3.1 Constraints Driving the Search	5
3.2 Benchmarking Methodology	5
3.3 Candidates Evaluated.....	5
3.4 Results	6
3.5 Why a Reasoning Model Was Rejected.....	6
3.6 Final Decision: A Two-Tier System.....	6
4. Features Ensuring Accurate, Current Answers	7
4.1 The Recency Bonus Function	7
4.2 Citation-Grounded System Prompt.....	7
4.3 Multi-Query Search	8
4.4 Local Caching.....	8
5. Testing and Validation.....	8
5.1 Search Layer Testing (Tavily Behavior).....	8
5.2 Query Interpretation and Search-Efficiency Testing	9
5.3 Response Quality and Grounding Testing	9
5.4 Manually Discovered Real-World Failures	9
6. Frontend.....	10
6.1 Key Features	10
6.2 API Contract	10
7. Deployment and Local Execution.....	10
8. Known Limitations	11
9. Conclusion	11

1. Project Overview

This project is a retrieval-augmented generation (RAG) chatbot designed to answer general knowledge questions, current events, politics, stocks, laws, and similar topics, using live web search rather than a static, pre-trained knowledge base. Unlike a conventional chatbot that answers purely from what a language model learned during training, this system searches the live web for every question, retrieves and ranks relevant sources, and generates an answer that is explicitly grounded in and cited to those sources.

The project was completed as part of a two-month remote internship, under three constraints set by the project supervisor:

- No paid APIs may be used anywhere in the system.
- No cloud deployment, the system must run entirely on-premise, on a single local machine.
- No fixed compute or GPU is provided; the choice of model size and architecture had to be independently researched and justified.

These three constraints shaped nearly every architectural decision described in this report, from the choice of a locally hosted LLM served via Ollama rather than a cloud API, to the model size research described in Section 3, to the fully local storage and caching layers described in Section 4.

1.1 High-Level Architecture

At a high level, a user question flows through five stages before an answer is returned:

1. Query Interpretation: The raw question is checked for ambiguity and converted into a focused search query.
2. Web Search, the interpreted query (and, when it differs meaningfully, the original question) is sent to the Tavily search API.
3. Retrieval & Reranking, returned sources are chunked, embedded, and ranked by semantic relevance to the question, with a recency adjustment applied.
4. For Answer Generation, the top-ranked source excerpts are passed to a locally hosted LLM (via Ollama) with a strictly citation-grounded system prompt.
5. Response & Storage, the answer, its sources, and the conversation are returned to the frontend and saved to local chat history.

The system is split into two halves: a Python/FastAPI backend that performs all of the above, and a React/Next.js frontend that provides the chat interface. Communication between the two happens entirely over localhost no part of the running application depends on any cloud service except for the Tavily search API call itself, which is the one external dependency required by the project's core premise of answering questions using live, current information.

1.2 Technology Stack

Layer	Technology	Notes
LLM Inference	Ollama	Runs locally; serves models via an OpenAI-compatible API at localhost:11434
Language Models	Llama 3.2 3B, Qwen2.5 1.5B Instruct	Open-weight, run entirely on CPU; see Section 3 for selection rationale
Web Search	Tavily API	Free tier; the one component that must reach the internet, by design
Embeddings / Reranking	sentence-transformers (all-MiniLM-L6-v2)	Runs locally on CPU, no external API calls
Backend	Python, FastAPI	Exposes a REST API consumed by the frontend
Frontend	React, Next.js, Tailwind CSS	Built with AI-assisted tooling (v0, reactbits.dev) and iteratively refined

Local Storage	SQLite	Two databases: a search-result cache and a chat-history store
---------------	--------	---

2. Backend Pipeline in Detail

This section walks through each backend module and what it is responsible for, in the order a request actually flows through the system.

2.1 Query Interpretation (query_interpret.py)

The first stage a question passes through is query interpretation, handled by a dedicated, lightweight LLM call. This step has two jobs: turning a natural, conversational question into a focused search query, and detecting when a question is too ambiguous to search for at all.

A key design decision here was to make this an extraction task rather than a rewriting task. Early experimentation showed that asking a small (1.5B-3B parameter) model to freely rewrite a question introduced a real risk of semantic drift, the model could drop a negation, merge unrelated concepts, or otherwise change the meaning of the question before it ever reached the search engine. Constraining the model to extract a structured set of keywords already present in the question, following a fixed template, proved far more reliable for models at this size.

The interpretation step always uses the smaller Qwen2.5 1.5B model, regardless of which model tier the user has selected for the final answer. Interpretation is a comparatively simple, template-following task, so there is no benefit to paying the latency cost of a larger model for this stage.

The interpreter's output follows one of two formats:

- **KEYWORDS:** <timeframe> <subject> <attribute>, a structured search query, extracted only from words already present in the question.
- **CLARIFY:** <question>, returned when the question is too vague to search for at all (for example, “who will win the election” with no election specified). In this case, no search is performed, and the clarifying question is returned directly to the user.

A specific, non-obvious problem encountered during development is worth documenting here: the word “last” is genuinely ambiguous in English, and this ambiguity caused search results to break during testing. “Who won the last F1 grand prix” means “most recent,” but “the last Malaysian Grand Prix” (a race discontinued from the calendar) means “final.” Search engines were found to sometimes interpret “last” in the second sense, even when the first was intended, returning results about a discontinued event instead of the most recent one.

2.2 Web Search (search.py)

Search is performed via the Tavily API, chosen specifically because it returns pre-extracted, cleaned page text alongside each result rather than raw HTML, removing the need to build and maintain a custom web scraper. Each search call requests results biased toward the “news” topic category, since the project's target questions are inherently about current events rather than static reference material.

Every search result is cached locally (see Section 4.4) before being returned, both to conserve the free-tier API quota and to speed up repeated testing of the same question during development.

2.3 Retrieval & Reranking (retrieval.py)

Raw search results are not passed directly to the language model. Each source's text is split into overlapping chunks, and both the chunks and the user's question are embedded using the all-MiniLM-L6-v2 sentence-transformer model, running entirely locally on CPU. Chunks are ranked by cosine similarity to the question, and the highest-scoring chunks are kept, discarding irrelevant portions of longer pages.

This step directly addresses two problems: first, that individual sources returned by web search are often long and only partially relevant, and second, that a small language model has limited capacity to usefully process large amounts of irrelevant context. Full details on the recency-bonus adjustment applied at this stage are given in Section 4.1, since it is one of the project's key accuracy-related features.

2.4 Answer Generation (llm.py)

The final, top-ranked sources are formatted into a numbered context block and passed to the selected Ollama model, along with a strict system prompt (reproduced in full in Section 4.2) that governs citation behaviour, conflict handling, and refusal when sources are insufficient. Generation is run at a temperature of 0.0 to prioritise consistent, literal reporting of source content over creative variation, appropriate for a system whose purpose is factual accuracy rather than open-ended conversation.

2.5 Caching and Chat History (cache.py, chat_history.py)

Two independent local SQLite databases support the system. A search cache stores Tavily results keyed by the exact query and parameters used, with a short freshness window (30 minutes) appropriate for a system answering questions about current events, alongside a separate, longer-lived housekeeping pass that periodically purges genuinely old, already-expired rows so the cache file does not grow indefinitely. A second database stores full conversation history, grouped by a conversation ID, with a 30-day automatic deletion policy, long enough to be useful for reviewing past questions, short enough that the local database does not grow without bound over the life of the project.

3. Model Selection Process

A structured benchmarking process was carried out before any model was adopted into the pipeline. This section documents that process and the evidence behind the final decision.

3.1 Constraints Driving the Search

- No GPU is available on the development machine (Intel integrated graphics only); the final system must run acceptably on the CPU.
- No paid model APIs are permitted, ruling out cloud LLM providers as the production inference path.
- The chosen model(s) must run locally via Ollama so that the final deployed system requires no cloud dependency for its core reasoning.

3.2 Benchmarking Methodology

Because no local GPU was available for development, model comparison was conducted using free GPU compute on Google Colab, with 4-bit quantisation applied to each candidate model. This served two purposes: it made it feasible to test multiple model sizes, including some larger than could ever run in practice on the target CPU-only deployment machine, and it approximated the quality of the Q4 GGUF quantisation that the models would actually run under once deployed via Ollama.

A structured, nine-case test set was built specifically around the behaviours this project depends on, not generic capability benchmarks, but tests of citation accuracy, conflicting-source handling, resistance to noise, and, most importantly, resistance to hallucination. The test cases are described in full in Section 5.2. Each candidate model was scored on the same test set, using the same system prompt structure as the pipeline itself, so that benchmark results would transfer meaningfully to real pipeline behaviour.

3.3 Candidates Evaluated

Two model families were evaluated across matching size tiers: Meta's Llama 3.2/3.1 (1B, 3B, and 8B) and Alibaba's Qwen2.5 Instruct (1.5B, 3B, and 7B). Llama's models require an Hugging Face access request and license agreement; Qwen2.5 is Apache-2.0 licensed and fully open, with no gating, a meaningful consideration for anyone reproducing this research without waiting on an approval process. A reasoning-focused family, DeepSeek-R1-Distil (1.5B, 7B, and 8B variants), was also evaluated as a third comparison point, given its strong general reputation.

3.4 Results

Model	Score	Notes
Llama 3.2 3B	100%	Clean, concise, correctly cited every test case
Qwen2.5 3B	88.9%	One miss; the failure was a minor under-citation, not a factual error
Qwen2.5 1.5B	77.8%	Two misses, both benign: correct facts stated without a citation tag
Llama 3.2 1B	44.4%	Frequently refused to answer, even when the correct source was present
DeepSeek-R1 (all sizes)	Varied, up to 100%	Strong scores, but see Section 3.5, a serious hallucination was observed

A separate, hardware-focused finding is worth recording: in early testing, a larger model (Llama 3.1 8B) appeared dramatically slower (roughly 105 seconds per response) than smaller models. Once every model was tested under consistent 4-bit quantisation, that gap almost disappeared. The original slowdown was traced to the larger model partially exceeding available GPU memory and silently falling back to slower CPU-offloaded execution, not to its size alone. This is recorded here because it materially changed the size-versus-latency conclusion: once tested fairly, an 8B model bought negligible accuracy improvement over 3B at a meaningfully higher resource cost, with no benefit to the tasks this project depends on.

3.5 Why a Reasoning Model Was Rejected

DeepSeek-R1's distilled models scored competitively, including a perfect score at the 7B size. However, closer inspection of individual responses, reading the model's full reasoning trace, not just its final score, revealed a serious failure mode not visible in the aggregate number. In the hallucination-resistance test case (a question about a deliberately fictitious company), the smallest DeepSeek-R1 variant did not simply refuse to answer, as the correct behaviour requires. Instead, its internal reasoning fabricated a nonexistent supporting source, invented a ranking list that was never present in the provided context, and used that fabricated source to construct a confident, wrong answer.

This distinction matters significantly for a project whose entire purpose is answering only from verified, cited sources: a model that occasionally invents supporting evidence to justify a guess is a fundamentally worse failure mode than a model that simply says it doesn't know, even if both can produce an equally high raw benchmark score. This finding was the deciding factor in ruling out reasoning-style models for this project, in favour of models that reliably default to refusal when uncertain.

3.6 Final Decision: A Two-Tier System

Rather than deploying a single fixed model, the system ships two selectable tiers, configurable via a single environment variable, allowing the choice to be based on the hardware of whoever is running it rather than fixed in advance:

Tier	Model	Rationale
lite	Qwen2.5 1.5B Instruct	Ungated, the smallest reliable option, minimal RAM footprint for constrained hardware
standard (default)	Llama 3.2 3B Instruct	Best benchmark result of any candidate tested; the recommended default

An 8B/70B-class “pro” tier was deliberately not included in the final system: the benchmarking in Section 3.4 found no measurable accuracy benefit over the 3B tier on this project's actual task once tested fairly, at a substantial cost in

memory and latency that directly conflicts with the project's constraint that the system must run acceptably on ordinary consumer hardware without a dedicated GPU.

4. Features Ensuring Accurate, Current Answers

This section documents the specific mechanisms built into the pipeline to keep answers grounded in real sources and biased toward the most recent, correct information, the core requirement of a general-knowledge chatbot whose subject matter (news, politics, markets) is constantly changing.

4.1 The Recency Bonus Function

A recurring failure was discovered during manual testing: semantic relevance alone is not a reliable signal for “what just happened.” A comprehensive, all-time reference table (for example, a full historical list of race winners spanning decades) can score as highly semantically relevant to a question like “who won the last F1 grand prix” as a genuinely current article about last weekend's race, because both discuss the same topic in similar language. Left uncorrected, this caused the system to confidently cite outdated information.

The fix implemented in `retrieval.py`, `source_recency_bonus()`, adds a scoring bonus on top of (not in place of) the semantic relevance score, based on how recent a source appears to be:

- A source whose `published_date` falls in the current year receives the largest bonus.
- A source from the previous year receives a smaller bonus.
- When no `published_date` is available (common with web search results), the function falls back to scanning the source's own text for four-digit years and applies a smaller bonus based on the most recent year mentioned.

Critically, this is implemented as an additive bonus layered onto the semantic relevance score, not a replacement sort key. An earlier version of this logic sorted sources purely by detected year, which introduced a new failure mode: a source could rank first purely for looking recent, even when it was not genuinely relevant to the question (for example, a listicle whose title happens to mention the current year as part of a “latest reads” sidebar). Combining both signals, rather than letting either fully override the other, was found to be necessary for correct results.

4.2 Citation-Grounded System Prompt

The language model's behaviour is governed by a single, carefully iterated system prompt, reproduced below as it exists in the final system:

"You are a factual research assistant. Answer in detailed, concise answers. All answers need to be derived from the provided sources. Rules:

- IMPORTANT: If the sources describe multiple different events over time, YOU MUST ONLY use the single most recent event based on the provided dates.
- NEVER combine, merge, or synthesise facts from different events or dates. Treat the newest source as the only valid truth.
- With the answer, give additional information necessary to understand the context.
- Cite every claim with a plain bracketed number. eg [1] or [2][3]. State the source you derive your answer from, and then your answer.
- Don't show reasoning, don't list ruled out sources, don't list how you got the answer, just give the answer and cite the source(s).
- If sources conflict, say so in one sentence and state which you're relying on and why.
- If sources don't cover the question, say so plainly.

- If sources only partially answer the question, present what's there and note the gap."

This prompt evolved considerably over the course of development, and several of the rules present directly trace back to specific failures found during manual testing (documented in Section 5.3), including an explicit instruction never to synthesize facts across different dated events, added after the system was observed doing exactly that, and an instruction not to narrate its own reasoning, added after early responses read as indecisive even when they ultimately reached the correct answer.

4.3 Multi-Query Search

For every question, two searches can be issued concurrently: one using the raw user question and one using the query interpreter's extracted keywords, when the two differ meaningfully. Results are merged and de-duplicated by URL before reaching the reranking stage. This provides a safety net, if the interpretation step ever produces a poor search query, the original question's own search results are still available, at the cost of additional Tavily API credits per question, a deliberate tradeoff given the project's zero-cost-API constraint.

4.4 Local Caching

Beyond the performance and quota benefits described in Section 2.5, caching indirectly supports answer consistency during development and testing: repeated identical questions return identical underlying source data within the cache's freshness window, which was essential for isolating whether a change in answer quality came from the prompt, the model, or simply different search results being returned on a different run.

5. Testing and Validation

Testing on this project occurred at three distinct layers, each capturing a different class of problems: the search layer in isolation, the query interpretation layer in isolation, and the full end-to-end pipeline's answer quality. This section documents representative test cases from each layer.

5.1 Search Layer Testing (Tavily Behaviour)

Before any retrieval or generation logic was built on top of it, the raw Tavily search wrapper was tested against four deliberately chosen query types, to understand its behaviour and limitations early:

Test Case	Query	What It Checks
Time-sensitive	"Current stock price of Apple today"	Whether published_date is reliably populated (finding: it frequently is not, so downstream logic cannot assume it is always present)
Ambiguous/broad	"latest news"	How unfocused results become without a specific topic directly motivates the later query-interpretation step
Obscure / fabricated	a fabricated company name's earnings report	Whether the search returns empty or irrelevant results without crashing the pipeline, and whether it silently returns confident-looking but unrelated results (finding: it returns similarly-named real companies rather than an empty result, which is why the hallucination-resistance rule in the system prompt cannot rely on an empty result list, it has to work from the model correctly recognizing that the returned sources don't actually mention the entity asked about)
Normal general knowledge	"Who is the current Secretary General of the United Nations"	Baseline sanity check confirms clean, relevant results absent any of the above edge cases

5.2 Query Interpretation and Search-Efficiency Testing

The query interpretation step (Section 2.1) was tested to confirm two properties: that it correctly avoids sending unnecessary or duplicate searches to Tavily, and that it correctly declines to search at all when a question is too ambiguous to search for meaningfully.

- Genuinely ambiguous questions, such as “who will win the election” with no election specified, correctly trigger the CLARIFY response path and no search is sent to Tavily at all, directly reducing unnecessary API usage for questions no search could usefully answer, regardless.
- Questions with a real, extractable subject are correctly converted into short, focused keyword sets rather than passed to search verbatim, for example, “what is going on with the letterboxd takeover” is reduced to a short subject-focused query rather than a full sentence.
- A specific regression case, “who won the last F1 grand prix,” was used repeatedly throughout development to test the “last” versus “latest” ambiguity described in Section 2.1, and served as the primary evidence for iterating that rule until it correctly preserved recency intent.
- Duplicate-query avoidance was verified directly: when the interpreted keyword query is identical to the raw question, only one search call is issued rather than two, and this was confirmed by inspecting cache log output during testing rather than assumed from the code alone.

5.3 Response Quality and Grounding Testing

The most extensive testing effort went into verifying that generated answers are actually grounded in their sources rather than merely appearing to be. A structured nine-case test set, run against every candidate model during the benchmarking process described in Section 3, covered the following behaviours:

Test Case	What It Verifies
Single-source citation	A basic factual question is answered with the correct fact and a correctly numbered citation.
Correct-source selection	When multiple sources are present, the model cites the one that actually supports its answer, not simply the first source provided.
Conflicting sources	When two sources directly disagree, the model states the conflict explicitly rather than silently picking one side.
Insufficient information	When no provided source actually answers the question, the model states that plainly rather than guessing.
Hallucination resistance	When asked about a fabricated entity not present in any real source, the model does not invent a plausible-sounding answer.
Noise robustness	The correct answer is found and cited even when several irrelevant, unrelated sources are mixed in alongside it.
Multi-source synthesis	A question requiring two separate facts from two separate sources is answered completely, citing both.
Messy real-world context	The model correctly extracts an answer from unstructured, navigation-cluttered scraped web text, not just clean prose.
Zero sources	When the search returns nothing at all, the model states that it lacks information rather than answering from general training knowledge.

5.4 Manually Discovered Real-World Failures

Beyond the structured benchmark, a number of real, organic failures were found and fixed through hands-on testing with genuinely ambiguous or difficult questions, several of which materially changed the system's design:

- A question about “Letterboxd’s takeover” returned entirely unrelated dictionary-definition pages for the word “what,” because the unfiltered question's most search-engine-friendly keyword was a common English word rather than the actual subject, which directly motivated building the query-interpretation step.

- A question asking who won “the last F1 grand prix” returned a race from 2020, then a race from 2017, before the recency-bonus and “last versus latest” fixes (Sections 2.1 and 4.1) resolved it. This was the single most persistent bug in the project and is documented in both sections, as it required fixes at two separate pipeline stages to fully resolve.
- A question referencing two fictitious election candidates was found to produce an overly verbose response that cited specific sources to “prove” the fictitious names did not exist, prompting a system-prompt rule that negative claims about a nonexistent entity should be stated plainly, without citation, since citing sources to prove an absence reads as more rigorous than it actually is.

6. Frontend

The frontend is a React and Next.js chat application, built with AI-assisted tooling (v0 for initial scaffolding, reactbits.dev for visual components) and then iteratively debugged and refined to match the backend's exact API contract.

6.1 Key Features

- A chat interface with streaming-style multi-stage loading indicators (“Searching the web...”, “Reading sources...”, “Writing answer...”) so the user understands what is happening during the several-second wait for a response.
- Inline citation markers rendered as clickable badges, which scroll to and highlight the corresponding numbered source card.
- A model tier switcher, allowing the user to choose between the lite and standard tiers described in Section 3.6 on a per-question basis.
- A chat history sidebar, allowing past conversations (stored server-side per Section 2.5) to be browsed and reopened.
- A cache-clearing control, allowing the user to manually flush the local search cache.
- A custom colour scheme (black, deep blue, and purple) with a glowing border effect applied to interactive elements, built using Reactbits.dev components.

6.2 API Contract

The frontend communicates with the backend exclusively over a local REST API (localhost:8000), with the primary exchange being a POST request to /chat containing the question, the selected model tier, and an ongoing conversation ID, and a response containing the generated answer, its structured source list, the interpreted search keywords used, and the conversation ID to continue the thread.

7. Deployment and Local Execution

In keeping with the project's no-cloud-deployment constraint, the system is designed to run entirely on a single local machine, with no component requiring a public-facing server. A single startup script (scripts/start.bat for Windows, scripts/start.sh for Mac/Linux) automates the full setup and launch process for a new user:

- Verifies that Python, Node.js, and Ollama are installed, with clear guidance if any are missing.
- Creates the Python virtual environment and installs backend dependencies on first run only.
- Prompts the user to add their free Tavily API key to a local .env file on first run only.
- Pulls both required Ollama models if they are not already present.
- Installs frontend dependencies on first run only.

- Starts both the backend and frontend servers and opens the application in the user's default browser automatically.

On every run after the first, the script skips the setup steps entirely and simply launches the already-configured application, so that repeated use requires nothing more than executing the script.

8. Known Limitations

- CPU-only inference on typical consumer hardware is meaningfully slower than cloud-hosted alternatives; response latency was an explicit, ongoing area of investigation and optimisation during development, tracked and explained to the project supervisor as a direct, expected consequence of the no-GPU, no-cloud constraints set for the project.
- The reranking model (MiniLM) is a small, general-purpose embedding model, and was found during testing to occasionally confuse different senses of the same word (for example, conflating “last” meaning “most recent” with “last” meaning “final” in unrelated source titles), a known limitation of small embedding models rather than a bug specific to this implementation.
- The system does not perform arithmetic or multi-fact computation over its sources by design, if answering a question would require combining multiple facts into a derived conclusion (for example, computing a list by subtraction across two source lists), the system is instructed to present the individual relevant facts rather than compute an answer, since this kind of implicit reasoning was found to be an unreliable and higher-risk task for models at this size.
- Web search quality for a given question is ultimately bounded by what the search API itself returns; a small number of genuinely obscure or extremely recent questions may not have adequate source coverage regardless of pipeline-side improvements.

9. Conclusion

This project set out to build a general-knowledge chatbot capable of answering current-events questions with verifiable, cited sources, under a specific set of constraints: no paid APIs, no cloud deployment, and an independently researched model choice. The final system meets all three constraints while implementing a genuinely multi-stage RAG pipeline, query interpretation, multi-query search, semantic reranking with a purpose-built recency adjustment, and citation-grounded generation, backed by a structured, evidence-based model selection process and an extensive record of real-world debugging that shaped both the retrieval logic and the system prompt in concrete, documented ways.

The most significant technical finding of the project was arguably not a feature but a caution: that a model's aggregate benchmark score can conceal a serious underlying failure mode, as demonstrated by the DeepSeek-R1 hallucination case in Section 3.5. That finding, more than any single feature, shaped the project's overall approach, prioritising a model that reliably admits uncertainty over one that scores marginally higher while occasionally fabricating supporting evidence, which is the single most important property a source-confirmed chatbot can have.